

Step-by-Step Guide: Implement CI/CD for ASP.NET Core Web API using GitHub Actions + Azure App Service

This guide is perfect if you already have an ASP.NET Core Web API project (like the ones you built earlier) and want to automatically build, test, and deploy it to Azure App Service every time you push to GitHub.

Prerequisites

- GitHub account + repository containing your ASP.NET Core Web API
- Azure account (free tier is fine)
- Azure App Service already created (Linux or Windows + .NET 8/6 recommended)
- Visual Studio 2022 or VS Code (optional, just for local dev)

Step 1: Prepare Your Azure App Service (One-Time Setup)

1. Go to <https://portal.azure.com>
2. Create → Web App
 - Resource Group: Create new or use existing
 - Name: **myapi-demo-2025** (this becomes your URL)
 - Publish: Code
 - Runtime stack: **.NET 8 (LTS)** or **.NET 6 (LTS)**
 - Operating System: Linux (cheaper) or Windows
 - Region: Choose closest
 - App Service Plan: B1 or higher (Free F1 doesn't support custom CI/CD well)
3. Click Review + Create → Create
4. Wait until deployment succeeds

Step 2: Get Azure Publish Profile (Required for Deployment)

1. In Azure Portal, go to your App Service → Overview
2. Click **Get publish profile** (top menu)
3. Download the **.PublishSettings** file → Save it securely (you'll use it once)

Step 3: Add Secrets to GitHub Repository

1. Go to your GitHub repo → Settings → Secrets and variables → Actions
2. Click **New repository secret** and add these 4 secrets:

Secret Name	Value (What to put)
AZURE_WEBAPP_NAME	Your App Service name (e.g., myapi-demo-2025)
AZURE_WEBAPP_PACKAGE_PATH	. (dot) → means publish from project root
AZURE_WEBAPP_PUBLISH_PROFILE	Open the downloaded .PublishSettings file → copy entire content and paste

Secret Name	Value (What to put)
<code>DOTNET_VERSION</code>	<code>8.0.x</code> (or <code>6.0.x</code> depending on your project)

Tip: Never commit the publish profile to code!

Step 4: Create GitHub Actions Workflow (The Main Part)

In your repo, create this folder and file:

```
.github/workflows/azure-webapp.yml
```

Paste this complete, production-ready workflow:

```
name: Build and Deploy ASP.NET Core Web API to Azure

on:
  push:
    branches:
      - main          # Trigger on push to main
  pull_request:
    branches:
      - main          # Also run on PRs (good for testing)

env:
  AZURE_WEBAPP_NAME: ${ secrets.AZURE_WEBAPP_NAME }
  DOTNET_VERSION: ${ secrets.DOTNET_VERSION }    # e.g., 8.0.x
  WORKING_DIRECTORY: .                          # Change if .sln is in subfolder

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup .NET SDK
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: ${ env.DOTNET_VERSION }

      - name: Restore dependencies
        run: dotnet restore
        working-directory: ${ env.WORKING_DIRECTORY }

      - name: Build
```

```

run: dotnet build --no-restore --configuration Release
working-directory: ${ env.WORKING_DIRECTORY }}

- name: Run Tests
  run: dotnet test --no-build --configuration Release --logger trx --
collect:"XPlat Code Coverage"
  working-directory: ${ env.WORKING_DIRECTORY }}

- name: Publish Artifact
  run: dotnet publish --no-build --configuration Release --output ./output
  working-directory: ${ env.WORKING_DIRECTORY }}

- name: Deploy to Azure Web App
  uses: azure/webapps-deploy@v3
  with:
    app-name: ${ env.AZURE_WEBAPP_NAME }}
    slot-name: production
    package: ./output
    publish-profile: ${ secrets.AZURE_WEBAPP_PUBLISH_PROFILE }}

- name: Deployment Success Notification
  if: success()
  run: echo "API deployed successfully! 🚀 https://${ env.AZURE_WEBAPP_NAME
}}.azurewebsites.net/swagger"

```

Step 5: Commit and Push → Watch Magic Happen

```

git add .github/workflows/azure-webapp.yml
git commit -m "Add CI/CD pipeline with GitHub Actions"
git push origin main

```

Now go to your repo → Actions tab → You'll see the workflow running!

Step 6: Verify Deployment

Once it says "All jobs have passed":

- Open: <https://your-app-name.azurewebsites.net/swagger>
- Your API should be live with the latest code!

Bonus: Recommended Improvements (Add Later)

Feature	How to Add Later
Run only on tags (for production)	Change trigger to <code>on: push: tags: ['v*']</code>
Separate staging slot	Add slot deployment + swap workflow
Cache NuGet packages (faster)	Add <code>actions/cache@v4</code> step

Feature	How to Add Later
Send email/Slack on failure	Add notification steps

You now have a complete, professional CI/CD pipeline!

Every time you push to `main`, your ASP.NET Core Web API will be automatically:

- Built
- Tested
- Deployed to Azure
- Live in seconds